

A hybrid modified-NSGA-II VNS algorithm for the Multi-Objective Critical Disruption Path Problem

Donatella Granata^{b,c,1}, Antonino Sgalambro^{a,b,*,1}

^a University of Sheffield, Management School, Conduit Road, S10 1FL, Sheffield, United Kingdom

^b Istituto per le Applicazioni del Calcolo "Mauro Picone", National Research Council, via dei Taurini 19, 00185 Rome, Italy

^c Dipartimento di Scienze Matematiche, Fisiche e Informatiche, Plesso di Matematica, Università degli Studi di Parma, Parco Area delle Scienze 53/A, 43124 Parma, Italy

ARTICLE INFO

Keywords:

Networks
Critical disruption path
Mixed integer programming
Multiple objective optimisation
Metaheuristics

ABSTRACT

This paper considers a Multiple Objective variant of the Critical Disruption Path problem to extend its suitability in a range of security operations relying on path-based network interdiction, including flight pattern optimisation for surveillance. Given a pair of nodes s and t from the network to be monitored, the problem seeks for loopless $s - t$ paths such that, within the induced subgraph obtained via deletion of the path, the size of the largest connected component is minimised, the number of connected components is maximised, while concurrently reducing as much as possible the cost of such disruption path. These three objectives are possibly in conflict with each other, and the scope of this work is to allow for an efficient and insightful approximation of the Pareto front, looking for a trade-off between costs and effectiveness to secure the most convenient paths for security and surveillance operations. We first introduce and formulate the Multi-Objective Critical Disruption Path Problem (Multi-Objs-CDP) as a mixed integer programming formulation (MO-CDP), then we propose an original evolutionary metaheuristic algorithm hybridising modified-NSGA-II and VNS for finding an approximation of the Pareto front, as well as a procedure securing the efficient generation of a high quality pool of initial solutions. The experimental performance of the proposed algorithm, as compared with a variety of competing approaches, proves to be fully satisfactory in terms of time efficiency and quality of the solutions obtained on a set of medium to large benchmark instances.

1. Introduction: the role of critical disruption paths in network surveillance and security

Many real-world problems arising in safety and security can be accurately represented through networks, enabling the adoption of optimisation methods to support better decision making, also in presence of multiple and often conflicting criteria. Network modelling is typically utilised to predict and evaluate the behaviour of a system, to identify the most vital (also called vulnerable or critical) components of a network, via measuring the rules governing individual nodes, arcs, or associated substructures, whose failure will prevent the functionality of the network as a whole. In the last decades a growing scientific attention has been devoted towards a specific class of problem, referred to as Network Interdiction, where one is concerned with studying those disruption interventions which are expected to induce a residual network as damaged as possible. Work in the field of network interdiction dates back to Harris and Ross (1955) and, over the years, network interdiction models have been increasingly applied in many

different areas (Wollmer, 1964; Ball et al., 1989; Wood, 1993; Lim and Smith, 2007; Smith and Song, 2020). These problems are relevant from both a protection and an interdiction perspective. In the latter case, the interdiction is an attack to the network that leaves the network fragmented or disconnected and the interdictor chooses his optimal strategy to attack a given network, or a part of it. In the former, the defender identifies which network components are the most critical ones for maintaining the integrity of the network, and therefore should be protected or reinforced. Network interdiction problems, besides focusing on scenarios that cause the network to become non-operational after the failure of some nodes or arcs, are also aimed at measuring the network's communication capability or level of degradation (Albert et al., 2000; Mishkovski et al., 2011; Holme et al., 2002). The definition of network failure varies, but it typically involves either nodes or arcs or specific topological substructure that may fail under certain conditions or specific attacks. It is often advisable to evaluate the number of necessary disruptive events to experience given levels of disruption

* Corresponding author at: University of Sheffield, Management School, Conduit Road, S10 1FL, Sheffield, United Kingdom.

E-mail addresses: donatella.granata@cnr.it (D. Granata), a.sgalambro@sheffield.ac.uk, antonino.sgalambro@cnr.it (A. Sgalambro).

¹ Authors contributed equally.

in the target network, in order to assess its vulnerability. This led to studies on how to lower the overall pairwise connectivity as a measure of the network performance (Dinh et al., 2010). Furthermore, examples of arc removal interdiction models have been introduced in Israeli and Wood (2002), Royset and Wood (2007), whereas interdiction models based on node removal have been broadly studied by Ventresca (2012), Ventresca and Aleman (2014a,b), Summa et al. (2011), Arulselvan et al. (2007, 2009), Cohen et al. (2003), Zhou et al. (2006), Shen et al. (2012), Lalou et al. (2018) and Walteros et al. (2019), and the shortest path interdiction problem has been considered in Israeli and Wood (2002). In other variants of interdiction problems, the aim of nodes and/or arcs removal is to get minimum weight on specific structural and topological properties in the remaining graph (Smith and Song, 2020), including cliques (Furini et al., 2019; Mahdavi et al., 2014) and maximum matching (Zenklusen, 2010). When planning complex security operations, an insightful analysis and assessment of network's vulnerability and control is sometimes best achieved by seeking those loopless paths in the network whose removal maximally impedes network operability or maximises its disruption. These are referred to as Critical Disruption Paths (CDPs) and were firstly introduced in Granata et al. (2013). CDP's applications arise for instance when defining optimal patterns for surveillance purposes, thus including: flight patterns for military surveillance and reconnaissance missions, realised through remotely piloted or traditional aircraft, optimal pedestrian or cycling paths for surveillance and control workforce.

With the aim of enhancing the outcome of the surveillance activity for network control purposes, different measures have been adopted and investigated as an objective for the CDP problem, including:

- minimising the size of the largest connected component in the induced subgraph obtained via deletion of the CDP: this goal will lead to containing the residual risk of adverse activities taking place in the residual network after the surveillance intervention (Granata et al., 2013);
- maximising the number of connected components in the induced subgraph obtained via deletion of the CDP: this goal is aimed at decreasing the connectivity of isolated components after the disruptive action (Granata and Sgalambro, 2016).

A preliminary bi-objective study presented in Granata and Sgalambro (2016) showed how a combined use of both objective functions can yield a range of non-dominated solutions to choose from, while evaluating the most appropriate intervention to be implemented for practical purposes. Those experiments also revealed how the cost allowed for the critical path impacts on the trade-off between such different and often conflicting goals, thus revealing the presence of a prominent research gap: how to design an efficient algorithm to approximate the Pareto front for multiple-objective variants of the CDP with conflicting relevant objectives. Furthermore, the cost of the CDPs clearly reveals a feature to be considered as a major decision making driver, as it influences both: the capability to implement surveillance operations within given limitations in times, budget and vehicle endurance, and the chance of finding proper trade-off between conflicting network disruption measures. In this work we contribute to bridge these gaps, as follows.

First, we introduce the Multiple Objective Critical Disruption Path problem where, given a pair of nodes s and t and the network to be monitored with weights associated to arcs, one seeks for the loopless $s-t$ paths such that, within the induced subgraph obtained via deletion of the path, the size of the largest connected component is minimised, the number of connected components is maximised, and the cost of such a disruption path is concurrently minimised. The three considered objectives are in conflict with each other, and the scope of this work is to foster an efficient and insightful approximation of the Pareto front of this optimisation problem, looking for the trade-off between costs and effectiveness in the solutions, thus supporting the decision maker

at identifying the most suitable paths for security and surveillance operations.

Secondly, we propose an original evolutionary metaheuristic algorithm which hybridises modified-NSGA-II and VNS for approximating the Pareto front of the considered Multi-objective Critical Disruption Path problem. To the best of our knowledge, this is overall the first solution approach proposed for a multiple-objective CDP, and its performance is compared to a variety of rigorous competing approaches, again proposed and implemented in this work.

As a further contribution, we propose an original polynomial time procedure aimed at generating a pool of tailored feasible solutions by identifying in polynomial time nodes which cannot belong to any CDP, thus at complementing and boosting the performance of the metaheuristic scheme. The experimental performance of the proposed algorithm proves to be fully satisfactory in terms of time efficiency and quality of the solutions obtained on a set of medium-to-large benchmark instances. The considered testbed is comprehensive and large enough to check and secure the scalability of the proposed method on any realistic size application. The remainder of our paper is organised as follows. In Section 2 we introduce formally the Multi-Objective Critical Disruption Path problem (Multi-Objs-CDP) providing a mixed integer programming formulation (MO-CDP). In Section 3 we discuss algorithmic strategies to restrict the search for Pareto efficient solutions as the generation of an initial pool of solutions. In Section 4 we present our Hybrid Multi-Objective Modified-NSGA-II Variable Neighbourhood Search (MO-NSGA-VNS) approach to solve efficiently the Multi-Objs-CDP problem. The experimental performance of the proposed MO-NSGA-VNS algorithm is presented in Section 5 on a set of medium to large benchmark instances, as compared to a variety of competing approaches, including two further NSGA-based algorithms and a scalarisation technique implemented to calculate non-dominated solutions by applying an off-the-shelf solver to the MO-CDP MIP model resolution. Some final remarks and further research avenues conclude the paper.

2. Problem statement

In this section we first introduce the Multi-Objective Critical Disruption Path Problem (Multi-Objs-CDP) and its Mixed-Integer formulation (MO-CDP). The Multi-Objective Critical Disruption Path Problem (Multi-Objs-CDP) proposed in this paper is stated as follows.

We are given a directed graph $\mathcal{G} = (V, E, w, s, t)$ with node and arc sets V and E of size n and m respectively and two special nodes: a source node $s \in V$ and a destination node $t \in V$. We assume w.l.o.g. that if $(i, j) \in E$ then also $(j, i) \in E$, and w_{ij} is the weight assigned to arc $(i, j) \in E$. Given any path ρ , we refer to $\mathcal{G}^\rho := (V^\rho, E^\rho)$ as the induced subgraph obtained via deletion of ρ from $\mathcal{G}$, $V^\rho := V \setminus V(\rho)$, $E^\rho := E \cap (V^\rho \times V^\rho)$, being $V(\rho)$ the subset of nodes of V which are included in path ρ .

Definition 2.1. The Multi-Objective Critical Disruption Path Problem (Multi-Objs-CDP) is defined as the problem of finding a simple loopless path ρ from s to t such that the following objectives are pursued:

- minimise the size of the largest connected component in $\mathcal{G}^\rho$;
- maximise the number of connected component in $\mathcal{G}^\rho$;
- minimise the cost of the path ρ .

The Multi-Objective Critical Disruption Path can be now formulated through the following multiple objective Mixed Integer Programming model. We term this model as MO-CDP.

$$\begin{aligned} \text{MO-CDP : } f_1 &:= \min \psi \\ f_2 &:= \max \sum_{i \in V \setminus \{s, t\}} d_i \\ f_3 &:= \min \sum_{i, j : (i, j) \in E} w_{ij} x_{ij} \end{aligned}$$

$$\text{s.t.} \quad \sum_{j:(i,j) \in E} x_{ij} = \sum_{j:(j,i) \in E} x_{ji} \quad \forall i \in V \setminus \{s, t\} \quad (1)$$

$$\sum_{i:(s,i) \in E} x_{si} = 1 \quad (2)$$

$$\sum_{i:(i,t) \in E} x_{it} = 1 \quad (3)$$

$$x_{ij} + x_{ji} \leq 1 \quad \forall (i, j) \in E, j > i \quad (4)$$

$$\sum_{i,j \in S: (i,j) \in E} x_{ij} \leq |S| - 1 \quad \forall S \subset V, |S| \geq 2 \quad (5)$$

$$y_{ii} = 1 - \sum_{l:(l,i) \in E} x_{li} \quad \forall i \in V \setminus \{s, t\} \quad (6)$$

$$y_{ij} \geq y_{ih} - \sum_{l:(l,j) \in E} x_{li} \quad \forall h, i, j \in V \setminus \{s, t\} : j \neq i, (h, j) \in E \quad (7)$$

$$\psi \geq \sum_{j \in V \setminus \{s, t\}, j \geq i} y_{ij} \quad \forall i \in V \setminus \{s, t\} \quad (8)$$

$$n \cdot d_i \leq n \cdot y_{ii} - \sum_{j \in V \setminus \{s, t\}: j < i} y_{ji} \quad \forall i \in V \setminus \{s, t\} \quad (9)$$

$$\psi \geq 0 \quad (10)$$

$$d_i \in \{0, 1\} \quad \forall i \in V \setminus \{s, t\} \quad (11)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in E \quad (12)$$

$$y_{ij} \in \{0, 1\} \quad \forall i, j \in V \setminus \{s, t\} \quad (13)$$

Among all the $s - t$ paths, here we are concurrently looking for the shortest ones (as required by the third objective function f_3) and the most disruptive ones, such that the arising connected components in the residual graph after the path removal present the largest amount of connected components with reduced size: this is obtained by including f_2 and f_1 as objective functions, respectively. The proposed model is an arc-based formulation requiring the selection of one simple $s - t$ path in the network, in the following indicated as Multi-Objs-CDP. The binary variables x_{ij} encode the decision on the choice of the CDP, and the binary decision variables y_{ij} take value 1 if and only if both nodes $i \in V$ and $j \in V$ belong to the same connected component after path extraction and 0 otherwise; y_{ii} has value 1 if node $i \in V$ is not belonging to Multi-Objs-CDP path and 0 otherwise. A non negative variable ψ is used to indicate the cardinality of the largest connected component arising in the residual graph. Binary variables d_i are used to count the number of connected components: as it is necessary to have a member of each component in the residual graph to represent that component, we use the node with the highest index as such one representative, hence d_i equals 1 if and only if node i is the rightful component representative, 0 otherwise. Constraints (1)–(5) ensure that variables x_{ij} identify a simple $s - t$ path, where each path node has one associated arc in and out as verified by the balance constraints (1). The complexity of sub-tour elimination constraints (5) is reduced by a separation mechanism and by constraints (4). Constraints (6) and (7) identify the connected components in the network: where two nodes $j \in V$ and $i \in V$ are forced to be in the same connected component, if j does not belong to the selected path and there exists a further node $h \in V$ connected via an arc to node j into the same connected component. Constraints (8) assign the largest connected component size value to variable ψ , whereas constraints (9) count the number of connected components. It is worth recalling how the CDP detection problem was proven in Granata et al. (2013) to be NP -complete by reduction from the Hamiltonian Path problem, thus characterising also the class of complexity of the Multi-Objs-CDP problem proposed in this paper.

2.1. Relevance of the MO-CDP for application purposes

The enhanced potential for application purposes unleashed by the multiple objective model above introduced, as compared to the single

objective variant of the CDP, can be better understood by observing the set of non-dominated solutions obtained on the small example network presented in Fig. 1. In this toy example we are given a graph $G = (V, E, w, s, t)$ with nodes $V = \{s, t, 2, 3, 4, 5, 6, 7, 8, 9\}$ and 27 arcs, and two special nodes: a *source* node $s \in V$ and a *destination* node $t \in V$. The weight w_{ij} assigned to each arc $(i, j) \in E$ is shown along the arc in the figure. Multi-Objs-CDP efficient solutions for the graph example (a) are presented in the following subfigures $\{(b),(d)\}$, $\{(c),(e)\}$, $\{(f),(j)\}$, $\{(g),(k)\}$, $\{(h),(l)\}$, $\{(i),(m)\}$, where each subfigure couple shows on the left the $s - t$ CDP path with the corresponding values of the three objective functions (f_1, f_2, f_3) and on the right a representation of the remaining connected components into residual graph $\mathcal{G}_i^\rho$ after the removal of CDP ρ_i as depicted into subfigure (i). We recall that the objective function f_1 minimises the size of the largest connected component in $\mathcal{G}^\rho$, f_2 maximises the number of connected component in $\mathcal{G}^\rho$ and f_3 minimises the cost of the path ρ . Let us assume that the CDP is utilised here to define the best flight pattern for an unmanned aerial vehicle (UAV) aimed at automated aerial surveillance. The single objective CDP would simply suggest (h) as a flight pattern, as a way to minimise the size of the largest component in the residual network (l) . Such a solution might require a surveillance pattern exceeding (or not) the endurance of the adopted UAV, depending on the amount of budget invested in purchasing the fleet for security operations. By using the MO-CDP model it is possible to explore the progressive growth in the surveillance quality while allowing increasing cost for surveillance paths, thus showcasing all the trade-offs between costs and solutions quality. Computing and comparing solutions which approximate the Pareto front of the MO-CDP will allow to appreciate, for each given level of intervention cost, the associated expected impact on security and surveillance operations, thus informing accurately the decision making process to identify the most convenient level of investment.

3. Generating an initial set of feasible solutions

With the twofold goal of generating a pool of good initial solutions and increase the efficiency in computing high quality solutions, we are interested in identifying quickly those nodes which result in suitable candidates for inclusion in a CDP, and in excluding all the others from our search process. Our approach is underpinned by the preliminary observation that a node which cannot be included in any simple $s - t$ path, can be excluded while seeking for feasible solutions. In the following we refer to these as unreachable nodes. In order to find such nodes, we design a polynomial-time pair node disjoint algorithm, denoted as *Pool-Init-Gen*, which draws upon a variant of the procedure presented in Suurballe and Tarjan (1984) and is applied on a modified network. In this section, we first briefly recall some major results from the literature on disjoint paths, then we describe the approach adopted in this paper for generating an initial pool of feasible solutions.

Finding disjoint paths: variants and complexity. The term k shortest disjoint paths is plainly interpreted as follows: given an undirected graph $G = (V, E)$ and k distinct pairs of nodes $(s_1, t_1), \dots, (s_k, t_k)$, the objective is finding whether there exist k pairwise disjoint paths $P_1, \dots, P_k$ such that P_i is a path from s_i to t_i , for every $1 \leq i \leq k$. One may consider several variants: directed or undirected, node or arc disjoint. The node disjoint path problem was shown to be NP -hard by Li et al. (1975). Fortune et al. (1980) proved that the directed version is NP -hard even if $k = 2$. Shiloach (1980) presented a linear $O(n \cdot m)$ algorithm that, given an undirected graph $G = (V, E)$ and nodes s_1, s_2, t_1, t_2 , determines whether or not G admits two node disjoint paths, one connecting s_1 to t_1 and the other one s_2 to t_2 . Eilam-Tzoref (1998) proved that directed or undirected and node or arc disjoint path problems are also NP -complete for arbitrary values of k even for planar graphs with unit arc-costs. But the author actually provided a polynomial algorithm for the case of $k = 2$ with positive arc-costs. Furthermore, in Tragoudas and Varol (1997) the problem of finding

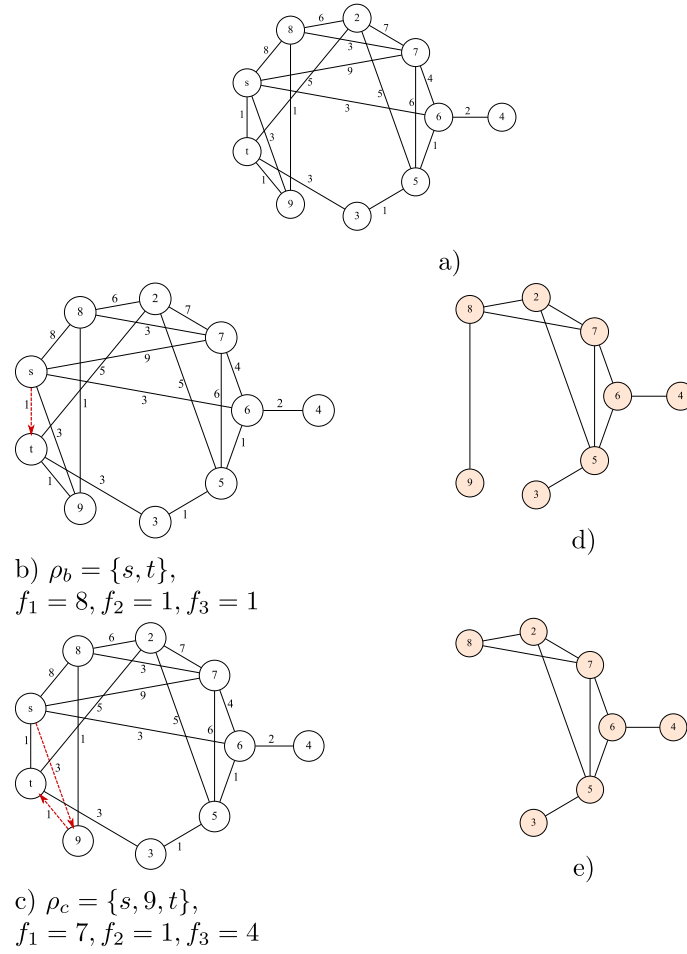


Fig. 1. Pareto front for a toy example.

a pair of length-bounded disjoint paths between nodes s and t of an undirected graph was proven *NP*-complete. The problem of finding two disjoint paths from s to t such that the length of the longer path is minimised was proven to be *NP*-complete on directed and undirected graphs (Li et al., 1990). Whenever the min-sum disjoint path variant is considered, namely where k disjoint paths with the total cost to be minimised are to be found, the problem is known to be polynomially solvable (Suurballe, 1974; Suurballe and Tarjan, 1984). Suurballe and Tarjan (1984) proved that given a directed graph $G = (V, E)$ with m arcs and with non negative weight assigned to each arc, finding a pair of shortest node/arc disjoint paths from s to a single sink t can be obtained in $O(m \cdot \log_{(1+m/n)} n)$ time. More results on the complexity of finding disjoint paths can be found in Vygen (1994) and Frank (1988). A recent heuristic for the computation of node disjoint path pair for any set of at least two intermediate nodes has been presented in Martins et al. (2017), which can be used on undirected and directed symmetric graphs. The paper also succinctly describes a procedure to obtain a min weight path visiting a specific node in undirected networks. In the next section, we propose and detail a novel algorithm which can be adopted on any graph. This routine is based on Suurballe's approach and considers one single intermediate node. Its worst-case complexity is bounded by $O(m + n \cdot \log(n))$ which stems from shortest path tree calculation.

3.1. Pool-Init-Gen algorithm construction

To generate a pool of good initial solutions and to timely identify those unreachable nodes which cannot belong to any CDP, we design a polynomial-time algorithm, namely *Pool-Init-Gen*, able to find

a min-hop $s - t$ path passing through a specific node i , by solving via an auxiliary modified network. This algorithm draws inspiration from Suurballe and Tarjan (1984), where finding a shortest $s - t$ pair of arc/node disjoint paths is used as a minimum-cost flow problem via Dijkstra algorithm. Its adaptation to this particular case is similar to the approach detailed in the introductory section of Martins et al. (2017).

The first step of *Pool-Init-Gen* algorithm consists in a building phase, to produce an auxiliary graph $G' = (V', E', s, t)$ from the original input graph $G = (V, E, s, t)$, which contains $|V'| = 2 \cdot |V - 2| + 2$ nodes and $|E'| = |V - 2| + |E|$. This building phase of G' incorporates the following finite steps:

- Each node $v \in V \setminus \{s, t\}$ is split in a pair of nodes denoted as v and v' .
- An arc $e = (v, v')$ is created connecting each couple of split nodes (v, v') .
- All the outgoing arcs of v are moved to be outgoing arcs of v' .
- A weight $w(u, v) = 1$ is assigned to each arc $(u, v) \in G'$.

The unitary weight value assigned in the last step to each arc in the auxiliary graph is instrumental at enabling the search for a min-hop path in the original graph. The effort of our algorithm is based on the idea that finding path p from s to t passing through a node i in the original graph G means looking for a shortest pair of node-disjoint paths, one from s to i and one from i' to t in the auxiliary graph G' . The *Pool-Init-Gen* involves these polynomial operations:

1. Find the shortest path tree $SPT(s)$ rooted at node s by running the Dijkstra's algorithm on the graph G' , and let call p_1 the found shortest path $(s \rightsquigarrow i)$ from source s to i .

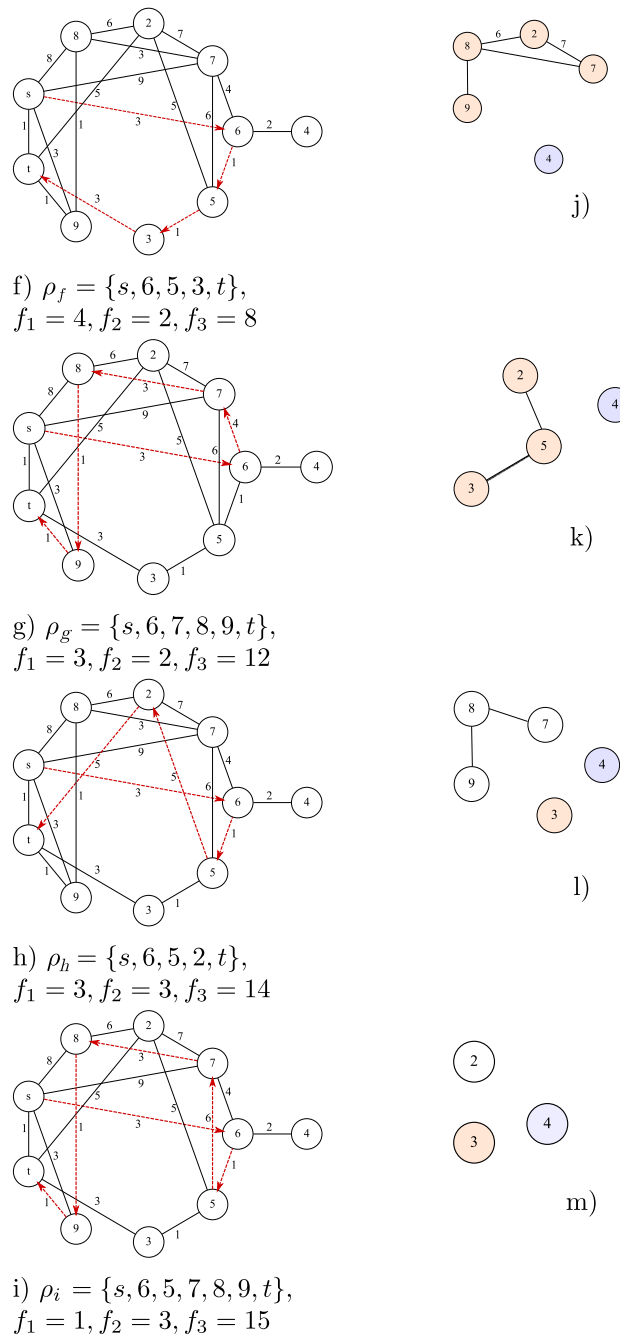


Fig. 1. (continued).

2. Modify the weight of each arc (u, v) in the graph by replacing its weight $w(u, v)$ by $w'(u, v) = w(u, v) - d(s, v) + d(s, u)$, being $d(i, j)$ the distance between any nodes i and j .
3. Create a residual graph G'^{p_1} formed from G' by reversing the direction of the zero weight arcs along path p_1 and by removing the arcs that are directed into s and, for all arcs $(u, v) \in |E| : v \in V(p_1) \setminus \{s, i\}$, set the weight $w(u, v) = |E|$.
4. Find the shortest path $p_2 = (i' \rightsquigarrow t)$ from the node i' to the sink t in the residual graph G'^{p_1} by running Dijkstra's algorithm. For each arc $(u, v) \in p_2$ remove arc (v, u) , if it exists, from both paths p_1 and p_2 .
5. Construct the $s - t$ path p concatenating p_1 and p_2 .
6. Modify p by shrinking any node splits previously operated during the building phase, thus preserving only those nodes and arcs which were originally part of the original graph G .

An example of auxiliary graph construction is presented in Fig. 2(b) applied to the dummy graph example represented in Fig. 2(a). So, Figs. 2(c)–2(f) depict the algorithm's steps in finding the path between the node s and t that passes through the node 1; both steps 1–2 of *Pool-Init-Gen* algorithm are depicted in Fig. 2(c), step 3 in Fig. 2(d), step 4 in Fig. 2(e) and steps 5–6 in Fig. 2(f).

Claim 1. If the cost of path p_2 , defined as $W(p_2) = \sum_{(u,v) \in E(p_2)} w(u, v)$, is greater than or equal to $|E|$, then there exist no $s - t$ path p passing through a given node i .

Proof. Seeking for the path p_2 , during the fourth step of the *Pool-Init-Gen* algorithm, means seeking for a simple path from i' to t in the residual graph G'^{p_1} , produced by step 3. In G'^{p_1} , all the arcs in $SPT(s)$ have weights equal to 0, and there are some arcs whose weight equals

$|E|$, namely, those arcs entering any node v belonging to $V(p_1) \setminus \{s, i\}$ (by algorithmic construction). If an arc (u, v) with $w(u, v) > 0$ is included in p_2 , this arc does not belong to the shortest path tree $SPT(s)$, because all the arcs in $SPT(s)$ have weights equal to 0. Furthermore, we can assert that the distance $d(i', v)$ from i' to v is bounded from above by $d(i', u) + w(u, v)$, namely, it holds $d(i', v) \leq d(i', u) + w(u, v)$, otherwise there would exist a shorter path to v via u , such that $d(i', u) - d(i', v) + w(u, v)$ is non negative and less than the longest path, which in turn can be at most equal to $n - 1$. Suppose for absurd, that $w(u, v) = |E|$: this would imply that during the construction of p_2 via Dijkstra's algorithm, one arc incident on $V(p_1) \setminus \{s, i\}$ has been included into the path p_2 , but this means that the final path p is not a feasible $s - t$ path solution for the graph G , and it is not passing through the node i , so a contradiction follows. $\square$

Furthermore, it is easy to show that G' contains a pair of arc-disjoint paths p_1 and p_2 , the former from s to i and the latter from i' to t , if and only if the original graph contains a path from s to t through node i .

3.2. Generating an initial population

Instead of simply using any preliminary random population generator as it often happens for genetic algorithms, we adopt iteratively the procedure presented above, with the scope of generating tailored initial solutions, taking into account the main features of the Multi-Objs-CDP, thus also securing increased chances to include non-dominated solutions in the starting population P_0 . Here we explain how the *Pool-Init-Gen* algorithm utilised the Algorithm (Section 3.1) to generate the initial population P_0 . The algorithm *Init_phase* is fed with a weighted directed graph $G = (V, E, w, s, t)$ with source s and destination t and returns as an output two sets: $UN \subset V$ and ND , filled with unreachable nodes and with a first pool of non-dominated solutions, respectively, with respect to the set of feasible solutions currently considered. *Init_phase* works by initially exploiting an implementation of the Dijkstra's algorithm for generating as its first solution a path which becomes in turn the first solution to be included in the set of non-dominated solutions ND (lines 1–2). The procedure *Pool-Init-Gen* is repeatedly executed upon each node i other than s and t (as detailed in Algorithm 1, lines 3–13). If a feasible solution is identified, the path p is included in the set ND , whereas if a feasible solution cannot be identified, the node i is labelled as unreachable and added to UN , as it is not possible to find any simple path from s to t through i .

Algorithm 1 *Init_phase*($G(V, E, w, s, t)$, ND , UN)

```

1:  $p \leftarrow \text{Dijkstra}(G(V, E, w, s, t))$ 
2:  $ND \leftarrow p$  {set of non-dominated solutions }
3: for all  $i \in V \setminus \{s, t\}$  do
4:   if  $\text{degree}[i] = 2$  and arcs  $(i, k), (k, i)$  exist then
5:      $\text{ADD}(UN, i)$ 
6:   end if
7:   Use Pool-Init-Gen algorithm for finding a path  $p$  passing through
   node  $i$ 
8:   if  $p$  is not a feasible solution then
9:      $\text{ADD}(UN, i)$ 
10:  else
11:     $\text{ADD}(ND, p)$ 
12:  end if
13: end for
14: return  $ND, UN$ 

```

4. Hybrid Multi-Objective Modified-NSGA-II Variable Neighbourhood Search (MO-NSGA-VNS)

In this section we present the original algorithmic procedure we designed *ad-hoc* to approximate the Pareto front for large size instances of the Multi-Objs-CDP problem. The adoption of bio-inspired

search paradigms has proved to be effective in solving many multi-objective optimisation problems (Emmerich and Deutz, 2018; Zitzler et al., 2004), as they are able to find multiple solutions simultaneously in a single execution. Methods such as Non-dominated Sorting Genetic Algorithm-II (NSGA-II) (Deb et al., 2002), Non-dominated Sorting Genetic Algorithm-III (NSGA-III) (Deb and Jain, 2014), Strength Pareto Evolutionary Algorithm 2 (SPEA2) (Zitzler et al., 2001), S-Metric Selection Evolutionary Multiobjective Optimisation Algorithm (SMS-EMOA) (Emmerich et al., 2005), and Multiobjective Evolutionary Algorithm based on Decomposition (MOEA/D) (Zhang and Li, 2007) have become extremely popular when it comes to solving multi-objective optimisation problems. NSGA-II (Deb et al., 2002) can be taken as a representative of Pareto-based approaches, the advantage to use this kind of method is the necessity to have few configuration parameters and the possibility to work well with a lot of objective functions meanwhile it is difficult to guarantee and measure the convergence of the solutions. SMS-EMOA (Emmerich et al., 2005) is a representative of the hypervolume indicator-based approaches: for these methods, it is possible to assess convergence at a high computational cost, since a set's hypervolume is measured in relation to a reference point and it equals the total size of the space dominated by the solutions in the set. MOEA/D (Zhang and Li, 2007) is a good representative of the decomposition-based approaches, for these methods is possible to incorporate various scalarisation methods but this requires some prior knowledge of the position of the Pareto front. Though, at a certain point it has become evident that a single metaheuristic is not sufficient to deal with the real world and large scale problems, so hybridised metaheuristics started to be presented in literature as reported by Gandibleux and Ehrgott (2005). The interaction among metaheuristics can take place at different levels, at low-level using specific functions from each metaheuristics or at high-level using a portfolio of metaheuristics for automated hybridisation (Talbi et al., 2012). Indeed, it has become popular the hybridisation between genetic algorithms and local search, that is also referred to as genetic local search, or *memetic algorithms* (Ishibuchi and Murata, 1996, 1998). Hybrid metaheuristics provide a more efficient behaviour and a higher flexibility. For instance, a two-phased approach based on the combination of a multi-objective evolutionary algorithms and single-objective techniques to solve Vehicle Routing Problems has been proposed by Jozefowicz et al. (2008) and two methods hybridised with the path relinking procedure, a Pareto ant colony optimisation algorithm and a variable neighbourhood search method by Schilde et al. (2009). A first hybrid NSGA-II and VNS for solving a bi-objective no-wait flexible workshop scheduling problem has been introduced in Asefi et al. (2014). Extended surveys have been provided in Emmerich and Deutz (2018), Zitzler et al. (2004), Talbi et al. (2012), Basseur et al. (2006) and Zavala et al. (2014). Our procedure is based on a hybridisation of NSGA-II and Variable Neighbourhood search (VNS), and we refer to this as the Hybrid Multi-Objective Modified-NSGA-II Variable Neighbourhood Search (MO-NSGA-VNS). Similarly to other population-based algorithms, the Pareto fronts (PFs) are formed and re-elaborated throughout the whole search process. At each step, solutions in the fronts are either kept or discarded according to a set of criteria and a new offspring population is generated. While many variants of NSGA algorithms generate preliminary random populations, we utilise the *Pool-Init-Gen* algorithm (see Algorithm Section 3.1) to create the initial population P_0 . Furthermore, instead of using basic operators such as mutation and crossover in order to generate new populations, in our algorithmic approach we adopt a modified Variable Neighbourhood Search scheme (see Section 4.1), which combines a variety of neighbouring structures and intensification procedures. A previous example of hybridisation between NSGA and VNS had been presented in the literature (Asefi et al., 2014): in this work, the algorithm was applied to a bi-objective problem, the initial input solution to VNS was a random offspring obtained by a mutation operator, the neighbours were applied to random jobs into layers and no intensification features were exploited. Before presenting

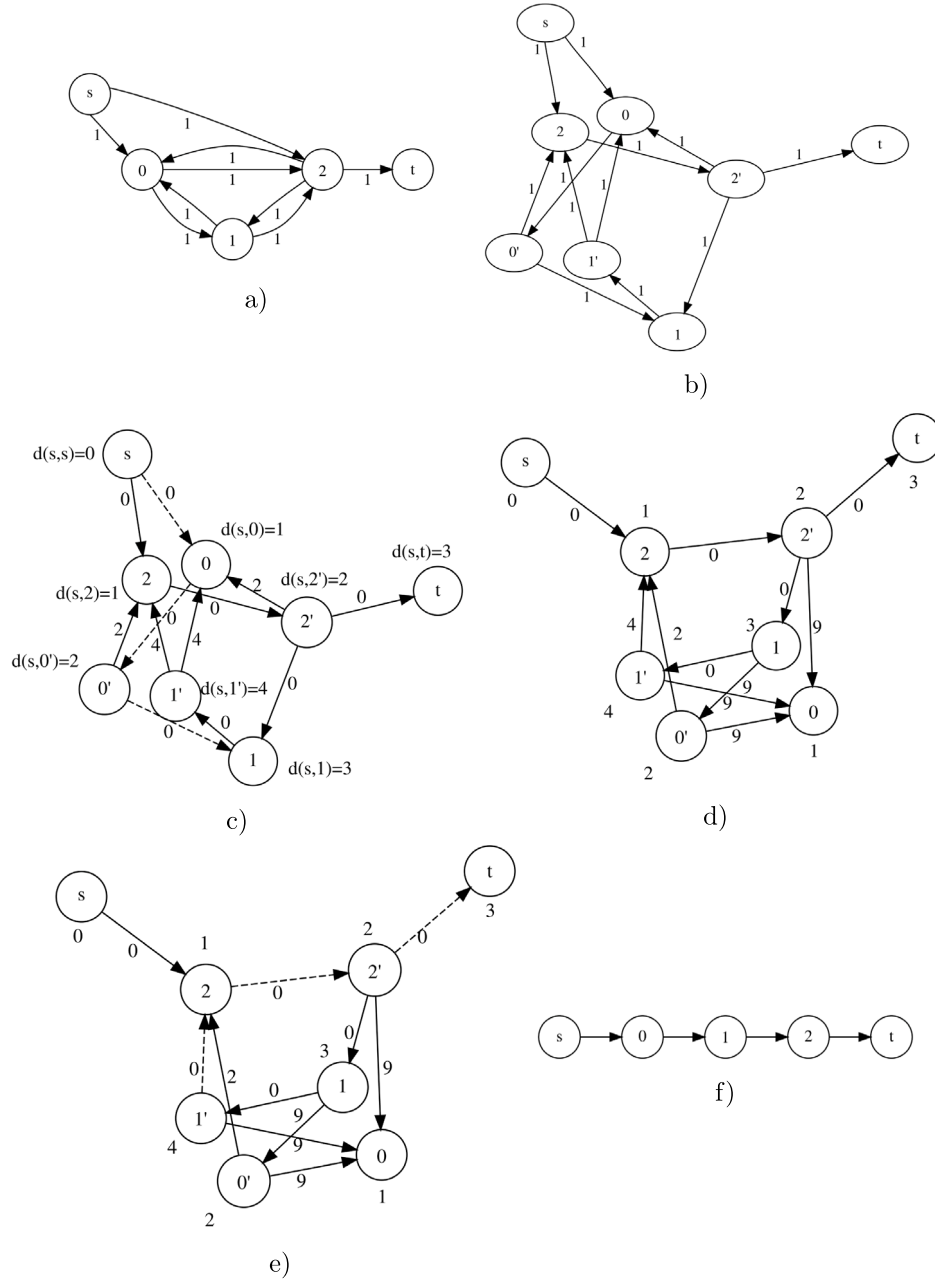


Fig. 2. Example of construction of shortest pair of node-disjoint paths s to 1 and from $1'$ to t .

the complete pseudocode description of MO-NSGA-VNS as applied to our problem in Algorithm 2, we describe some of the used support structures, parameters and useful recalling functions, to allow for a thorough understanding of the procedure scheme, as follows:

UN : set of unreachable nodes.

ND : set where all found non-dominated solutions are stored.

α : parameter related to neighbourhood $N_\alpha(p)$ (as presented in Section 4.1), increased at each iteration i .

β : parameter related to neighbourhood $N_\beta^{max}(p)$ (as presented in Section 4.1), increased at each iteration i .

$\hat{Q}$: population size limit.

$\hat{T}$: time limit in seconds.

$\hat{L}$: intensification iterations limit.

$ADD(ND, p)$: handler function defined to update the set ND . Given any solution p this function adds p to the current set of the non-dominated ND if and only if p is non-dominated by any other solution belonging to the set ND .

An initial population P_0 of size at most $\hat{Q}$ is generated using Algorithm 1, then a generational loop is repeated until time limit $\hat{T}$ is reached. At any i_{th} population generation, an offspring population Q_i of size $\hat{Q}$ is created using the *MOVNS* Algorithm (see Algorithm 3).

The new Pareto front population is obtained as $PQ_i = P_i \cup Q_i$ of size $2\hat{Q}$, this is divided into different non-dominated classes, or fronts, using the procedure *Fast_non_dominated_sort* presented by Deb et al. (2002). The domination counter is used to count how many solutions dominate the solution p , and it is adopted to determine whether a solution p belongs to a different non-dominated class, or front. The domination counter starts from zero for the first non-dominated front and reaches

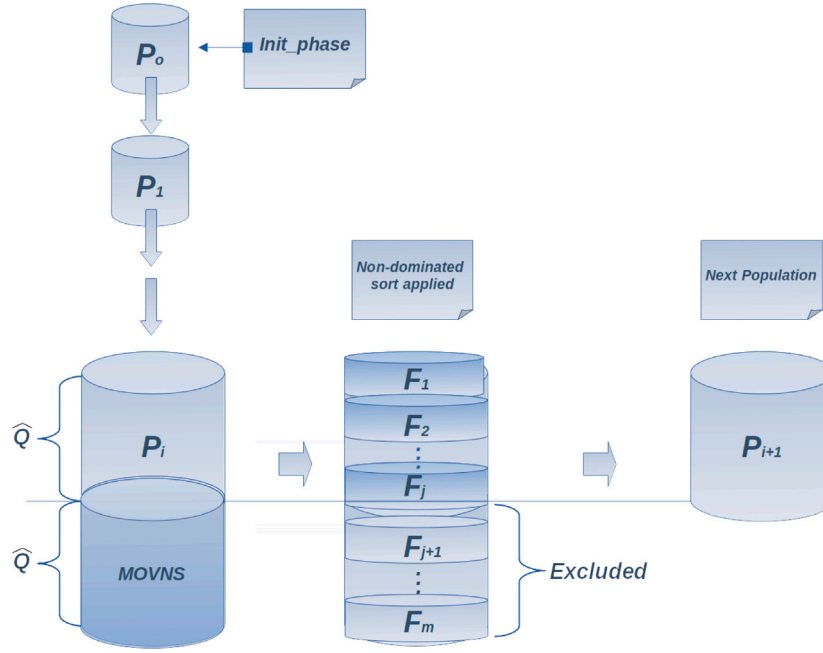


Fig. 3. Graphical sketch of the MO-NSGA-VNS procedure.

$|V| - 1$ for the last non-dominated front. Only the first $\hat{Q}$ individuals are kept to form the next generation, even if some of them can be members of the same last front. Note that no sub-procedures have been included to select and rank the last front. A graphical description of the whole evolutionary metaheuristic procedure is provided in Fig. 3.

Algorithm 2 NSGAII-MOVNS($G(V, E, w, s, t), \hat{Q}, \hat{T}, \hat{L}$)

```

1:  $UN \leftarrow \emptyset$  {Set of unreachable nodes of any path from  $s$  to  $t$ }
2:  $ND \leftarrow \emptyset$  {Set of the non-dominated Pareto solutions}
3:  $t = 0$  {Time counter}
4:  $i = 0$  {Iteration counter}
5:  $P_i \leftarrow \text{Init\_phase}(G(V, E, w, s, t), ND, UN)$ 
6:  $F \leftarrow \text{Fast\_non\_dominated\_sort}(P_i)$ 
7:  $Q_i \leftarrow \text{MOVNS}(G(V, E, w, s, t), P_i, \hat{Q}, \alpha, \beta, t, \hat{T}, ND, \hat{L})$ 
8: while  $t$  has not reached the time limit  $\hat{T}$  do
9:    $PQ_i \leftarrow P_i \cup Q_i$ 
10:   $F \leftarrow \text{Fast\_non\_dominated\_sort}(PQ_i)$  (Deb et al., 2002)
11:   $\text{ADD}(ND, F)$  {update the set of non-dominated solutions with those ones coming from frontiers stored in  $F$ }
12:   $P_{i+1} \leftarrow F[1 : \hat{Q}]$  {Take the first frontiers until to fill the next population  $P_{i+1}$  with at most  $\hat{Q}$  solutions}
13:   $Q_{i+1} \leftarrow \text{MOVNS}(G(V, E, w, s, t), P_{i+1}, \hat{Q}, \alpha, \beta, t, \hat{T}, ND, \hat{L})$ 
14:   $i \leftarrow i + 1$ 
15:  if  $\alpha > (|V|/2)$  then
16:     $\alpha = 1$  {re-initialise  $\alpha$  to 1}
17:  else
18:     $\alpha++$  {Increase  $\alpha$  until to reach half size of node number}
19:  end if
20:  if  $\beta > (|V|/2)$  then
21:     $\beta = 1$  {re-initialise  $\beta$  to 1}
22:  else
23:     $\beta++$  {Increase  $\beta$  until to reach half size of node number}
24:  end if
25: end while

```

4.1. Using a modified multi-objective variable neighbourhood search to generate the offsprings

Variable neighbourhood search, introduced by Mladenović and Hansen (1997), is a metaheuristic method whose search process draws

upon systematic changes of neighbourhood. The effectiveness of this procedure for solving single-objective optimisation problems has been broadly proved (Mladenović and Hansen, 1997; Hansen and Mladenović, 1997; Polacek et al., 2004). The first multi-objective VNS (MOVNS) was applied to a machine scheduling problem and proposed in Geiger (2008), differing from single-objective VNSs for having introduced two main arbitrary choices: the base unvisited non-dominated solution (starting point of the next neighbourhood search) and the used neighbourhood, both chosen at random from those available. This procedure has been further developed by Arroyo et al. (2011), where some non-dominated solutions have been constructed from partial results found by the inner procedures.

In our algorithm, we propose and adopt three parametric neighbourhoods $N_\alpha(x')$, $N_{\beta\max}(x')$, N_γ^{CROSS} and, at each iteration of the algorithm, all of the defined neighbourhoods are applied to a non-dominated solution p as randomly selected from the set P_i . Such neighbourhoods are:

$N_\alpha(p)$: obtained by removing up to α nodes from p and adding up to α nodes, completely at random.

$N_\beta^{\max}(p)$: obtained by removing up to β nodes from p and adding up to β nodes, where such nodes are chosen from those belonging to the maximum connected component. This choice is aimed at reaching better solutions by creating paths passing through nodes belonging to larger connected components.

$N_\gamma^{\text{CROSS}}(p)$: is obtained from p and a random non negative index γ , with $1 \leq \gamma \leq |p| - 1$ by concatenation of the subpaths p_γ and p_i , where $p_\gamma = \{v_s, \dots, v_\gamma\}$, (that is the subpath of p truncate at position γ) and $p_i = \{v_b, \dots, v_i\} \subset p \in ND$ with $v_b = v_\gamma$ (that is the subpath of a solution $\rho \in ND$ having v_γ as inner node).

A pseudo-code describing the complete modified Multi-Objective Variable Search (MOVNS) including the intensification phase is provided in Algorithm 3, which is also depicted in Fig. 4.

The aim of the intensification procedures is to add some new nodes to the current solution in order to shake the search procedure and escape from any local minima. This is realised by a recursive function, that is calling itself until no improvement can be performed on the best current solution (base case) or until one of the following terminating

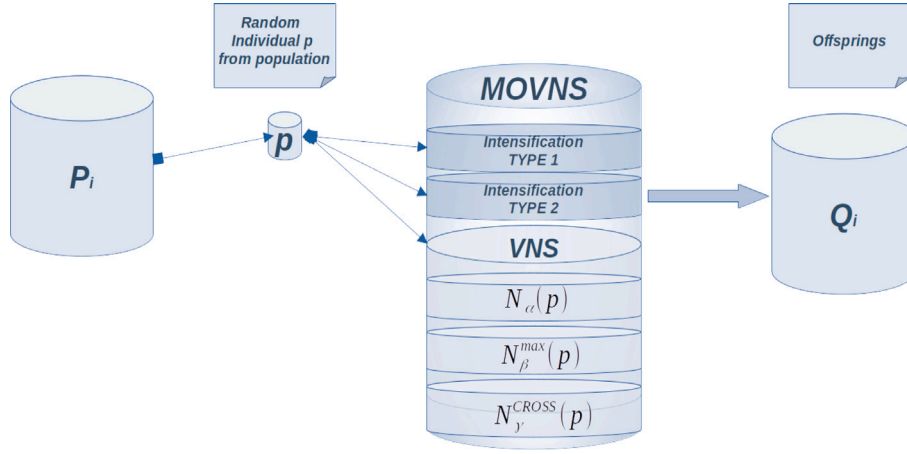


Fig. 4. Graphical sketch of the Algorithm 3.

Algorithm 3 MOVNS($G(V, E, w, s, t), P_i, \hat{Q}, \alpha, \beta, t, \hat{T}, ND, \hat{L}$)

```

1:  $Q_i \leftarrow \emptyset$  {set of offspring solutions}
2: while  $t \leq \hat{T}$  AND  $|Q_i| \leq \hat{Q}$  {Until time or Offspring size limits are reached} do
3:   Select randomly a solution  $p$  from the population set  $P_i$ 
4:   and mark  $p$  as visited
5:    $pd_1 \leftarrow \text{Intensification}(G(V, E, w, s, t), p, \hat{L}, \text{TYPE} \leftarrow 1)$ 
6:    $pd_2 \leftarrow \text{Intensification}(G(V, E, w, s, t), p, \hat{L}, \text{TYPE} \leftarrow 2)$ 
7:    $\text{ADD}(Q_i, pd_1)$  AND  $\text{ADD}(Q_i, pd_2)$ 
8:   Determine randomly a solution  $p'$  from  $N_\alpha(p)$ 
9:   for all  $p'' \in N_\alpha(p')$  do
10:    Evaluate the solution  $p''$ 
11:     $\text{ADD}(Q_i, p'')$  { $p''$  is non-dominated by any solution of offspring set  $Q_i$ }
12:   end for
13:   Determine randomly a solution  $p'$  from  $N_\beta^{\max}(p)$ 
14:   for all  $p'' \in N_\beta^{\max}(p')$  do
15:    Evaluate the solution  $p''$ 
16:     $\text{ADD}(Q_i, p'')$ 
17:   end for
18:   Determine randomly a non negative index  $\gamma$ , with  $1 \leq \gamma \leq |p| - 1$ 
19:   Determine randomly a solution  $p'$  from  $N_\gamma^{\text{CROSS}}(p)$ 
20:   for all  $p'' \in N_\gamma^{\text{CROSS}}(p')$  do
21:    Evaluate the solution  $p''$ 
22:     $\text{ADD}(Q_i, p'')$ 
23:   end for
24: end while
25: for all  $p \in Q_i$  do
26:    $\text{ADD}(ND, p)$ 
27: end for
28: return  $Q_i$ 

```

conditions is met: time limit $\hat{T}$ and depth of the recursion tree $\hat{T}$. We present two different procedures, both of them aimed at cutting a path p , which can be depicted by a sequence of nodes, in a specific position i , thus obtaining two subpaths $p_1 = s, \dots, i$ and $p_2 = j, \dots, t$, and filling the gap between nodes i and j with a new shortest path passing through a specific node u . The two procedures differ from how the node $u \in V$ is chosen (see Algorithm 4, and refer to lines 6 and 9). Hence a node u is selected at random from two sets, V^- and $V^{\max}$, for *Intensification TYPE 1* and *Intensification TYPE 2* respectively. The set V^- includes all reachable nodes except those belonging to the path p , whereas $V^{\max}$

is the set of all nodes belonging to the maximum size connected component. At each offspring generation, both intensification procedures are applied on a random solution chosen from the parent population, differently from Arroyo et al. (2011) where intensification is aimed at improving a partial VNS solution.

Algorithm 4 Intensification($G(V, E, w, s, t), p, \hat{L}, \hat{T}, \text{TYPE}$)

```

1:  $p' \leftarrow p$ 
2: while  $\hat{L}$  and  $\hat{T}$  limits are not satisfied do
3:   for  $i = 0$  to  $|V(p) - 2|$  do
4:     for  $j = i + 1$  to  $|V(p) - 1|$  do
5:       if  $\text{TYPE} = 1$  then
6:         pick a random node  $u$  from  $V^-$ ,
7:          $V^- \leftarrow V \setminus \{s, t, V(p_s, \dots, p_t), V(p_j, \dots, p_t), UN\}$ 
8:       else
9:         pick a random node  $u$  from  $V^{\max}$ ,
10:        {
11:           $V^{\max} \leftarrow CC \setminus \{UN\}$  where  $|CC|$  is the maximum connected component obtained removing  $p$  }
12:        end if
13:         $r' \leftarrow \text{Dijkstra}(G(V \setminus V(p), E \setminus E(p), w, s, t), i, u, j)$  {
14:          that is a shortest i-j path passing through node  $u$ , where  $i$  is the source and  $j$  is the sink }
15:        if  $r' \neq \emptyset$  then
16:           $p' = (p_s, p_1, \dots, p_t = r_i, \dots, u, \dots, r_j = p_j, \dots, p_t)$ 
17:        end if
18:        if  $p' < p$  then
19:           $\hat{L}++$  // increment to handle the recursion tree depth
20:           $\text{Intensification}(G(V, E, w, s, t), p', \hat{L}, \hat{T}, \text{TYPE})$  //recursive case
21:           $\text{ADD}(ND, p')$ 
22:        else
23:          return  $p'$  //case base
24:        end if
25:      end for
26:    end while
27:  return  $p'$ 

```

5. Testbed and computational experiments

In this section, we present and analyse the results of the articulated computational experience that has been developed in this work. The scope of the experiments is assessing the efficiency of the MO-NSGA-VNS algorithm and its efficacy at approximating the Pareto front of the

Table 1

Performance comparisons between MO-CDP and MO-NSGA-VNS. Column 1 describes the instance triplet (n, m, o) . Columns 2 and 4 present the number of non-dominated solutions NDs for both algorithms: MO-CDP and MO-NSGA-VNS. Running times are reported into columns 3, 5 and 6.

(n, m, o)	MO-CDP		MO-NSGA-VNS		
	NDs	Time (s)	NDs	Min(Time(s))	AVG(Time(s))
(40, 156, 0)	16	25816.76	33	1133	3024.15
(40, 156, 1)	11	19098.29	21	0	2947.00
(40, 780, 0)	9	153529.76	29	0	2461.55
(40, 780, 1)	13	189728.45	26	0	2093.81
(40, 1404, 0)	17	246153.33	29	2	2333.97
(40, 1404, 1)	14	260671.31	27	126	1578.22
(50, 245, 0)	20	39137.43	39	0	2705.69
(50, 245, 1)	17	27888.02	16	7	2826.88
(50, 1225, 0)	9	123010.44	27	1	2792.30
(50, 1225, 1)	6	139596.33	26	1822	3147.50
(50, 2205, 0)	–	–	22	1365	3126.77
(50, 2205, 1)	–	–	28	315	2052.75
(60, 354, 0)	23	81649.94	48	196	1470.44
(60, 354, 1)	10	63908.25	54	182	2120.20
(60, 1770, 0)	–	–	40	449	2462.05
(60, 1770, 1)	–	–	48	336	1868.25
(60, 3186, 0)	–	–	31	409	2767.23
(60, 3186, 1)	–	–	25	0	2744.80
(70, 483, 0)	16	184657.06	53	49	1765.02
(70, 483, 1)	28	153343.14	68	119	1602.93
(70, 2415, 0)	–	–	32	5	2288.56
(70, 2415, 1)	–	–	44	693	2602.36
(70, 4347, 0)	–	–	22	0	3018.64
(70, 4347, 1)	–	–	27	748	2596.74
(80, 632, 0)	5	48229.90	21	0	2412.76
(80, 632, 1)	12	169140.29	18	20	1533.89
(80, 3160, 0)	–	–	46	82	1673.41
(80, 3160, 1)	–	–	50	0	2306.84
(80, 5688, 0)	–	–	14	1457	2325.43
(80, 5688, 1)	–	–	22	3	2603.00
(90, 801, 0)	1	63118.42	26	19	2628.27
(90, 801, 1)	1	52530.52	19	0	3055.53
(90, 4005, 0)	–	–	26	961	2793.58
(90, 4005, 1)	–	–	26	0	2903.46
(90, 7209, 0)	–	–	21	1745	2753.76
(90, 7209, 1)	–	–	30	0	2899.10

Multi-Objective Critical Disruption Path Problem. We first provide here a description of the adopted testbed and of the range of different methods we considered in order to assess the performance of our algorithm. Then in Section 5.1 the results of the computational experiments are presented and discussed.

Computational testbed. A large set of increasing size random instances has been generated uniformly distributed as a test bed, overall made up of two different classes of instances:

- six groups of medium size, with a number of nodes $n = 40 + 10 \times \kappa$, $\kappa \in [0, 1, \dots, 5]$
- ten groups of large size instances, with a number of nodes $n = 100 + 100 \times \sigma$, $\sigma \in [0, 1, \dots, 9]$.

For both classes, the number of arcs is $m = n \times (n - 1) \times \rho$ where $\rho \in \{0.1, 0.5, 0.9\}$ and, for each pair (n, m) , two distinct randomly generated instances are created and referred to by using the notation (n, m, o) , with $o = \{0, 1\}$, to denote the instance occurrence. The algorithm for instance generation primarily consists of two phases: the connecting phase, with at most $n - 1$ steps, where arcs are iteratively generated at pseudo-random connecting one node already inserted in the building graph and the other one not yet connected, until none of the nodes are left out; the building phase, where all other arcs are generated at pseudo-random, by iteratively selecting origin and destination in V with uniform probabilities, until the required target number of arcs m is reached. Furthermore, for each arc e a weight l_e is generated at random with a uniform distribution in the interval $[1, n]$. The MO-NSGA-VNS

was coded in ANSI C++-14. All the computations have been performed on an Intel(R) Xeon(R) CPU E5-2680 v2 @ 2.80 GHz with 16 GB of RAM. The following parameters were adopted:

$\hat{T} = 3600$ s, as a computational time limit.

$\hat{L} = 100 * |E|$, with a maximum value of 10000, (i.e., the recursion tree depth cannot exceed the value of 10000) as an intensification depth parameter, used in the Intensification procedures (see Algorithm 4).

$\hat{Q} = 20$, as an offspring set size.

Note that the parameters $\hat{L}$ and $\hat{Q}$ have been set by following the results of a preliminary calibration phase based on the execution of the algorithm on a sample of instances.

Benchmarking against scalarisation technique. In order to assess quality and exhaustiveness of the MO-NSGA-VNS at generating the Pareto front, a comparison with the results of a multi-objective scalarisation technique, implemented by using a state-of-art off-the-shelf solver, is also provided in this paper for the class of medium size instances. To this aim, several convex combinations of the three objective functions were considered and experiments conducted at varying weight coefficients λ_i , $i = 1, 2, 3$, in the range $[0, 1]$ with such interval divided in 100 steps for each dimension, such that $\sum \lambda_i = 1$ for each triple of weight coefficients considered. As the goal of this specific comparison is concerned with challenging the capability of the MO-NSGA-VNS to produce a comprehensive Pareto front, rather than its computational efficiency, each experiment was executed by using IBM ILOG CPLEX 12.9 on the same machine without any time limitation in order to get the largest possible number of solutions, although clearly not exhaustive. The proposed model has been implemented using ANSI C++ and Concert Technologies libraries, using Cplex callbacks to cope with subtour elimination constraints (5). Thus, the set of non-dominated solutions $ND(\text{MO-CDP}) = \bigcup_{\lambda_i} ND_{\lambda_i}(\text{MO-CDP})$ found through implementing and solving the (MO-CDP) model by CPLEX consists in the union of all non-dominated solutions found whilst varying the convex weight λ_i combinations through the above described scalarisation technique.

Benchmarking against evolutionary algorithms. In order to test also time-efficiency and effectiveness of the MO-NSGA-VNS algorithm, two further comparisons against standard evolutionary algorithms were developed, through the implementation of the standard Multi-objective Evolutionary algorithms: *basic-NSGA* and *subpaths-NSGA*. The *basic-NSGA* is based on the classic implementation of NSGA-II where the mutation operator is obtained by replacing a node with a new one, as follows: given a solution path $p = \{s, \dots, i - 1, i, i + 1, \dots, t\}$, a random node i is substituted by another random node j as it is expected within the *basic-NSGA* algorithmic framework. As this simple mutation operator often does not guarantee the existence of another valid path, we also introduced *subpaths-NSGA* algorithm, whose main idea is to replace a node i by a simple sub-path $\{i - 1, \dots, i + 1\}$, created between the adjacent nodes $i - 1$ and $i + 1$ whose nodes are not belonging to the original path p . We use the *Intensification* procedure of *TYPE 1* (see Algorithm 4) as operator to find the best fitting subpath.

Adopted benchmarking metrics. non-dominated

$$NDsbyR(ND, R) = |nd \in ND | \nexists r \in R : r < nd \quad (14)$$

$$NDsRbyR(ND, R) = \frac{NDsbyR(ND, R)}{|ND|} \quad (15)$$

The reference set R is defined as a collection of candidate solutions with respect to which we can compare two algorithms. In other words, these metrics estimate, respectively, the amount of solutions in the set ND which are dominated by any solution belonging to the reference set R , expressed as absolute number and percentage, respectively.

5.1. Discussion of the results

The results of the computational experiments are now presented and discussed, focusing on the performance of MO-NSGA-VNS algorithm as measured against the Pareto front in terms of quality of the obtained solutions by adopting a variety of convergence-diversity criteria. Experiments are presented as follows. Firstly we present the comparison between the results obtained by MO-NSGA-VNS and those provided by the scalarisation technique approach on the (MO-CDP) model. Table 1 is aimed at comparing the overall number of non-dominated solutions provided (NDs) and the required CPU times, against each distinct instance. Recall for each pair (n, m) a couple of instance occurrences are generated, denoted with two triplets (n, m, o) , with $o = \{0, 1\}$. Computational times for the MO-CDP scalarisation technique express the sum of all computational times needed to solve each convex combination obtained at varying λ_i into the objective function, where no time limit was applied. As regards MO-NSGA-VNS, $Min(Time)$ and $AVG(Time)$ report respectively the time when the first solution was found and the average time when non-dominated solutions ND were identified, recalling how a time limit of 3600 seconds was set for this case. A value of the $Min(Time)$ column equal to 0 suggests therefore how at least one solution among the final set of non-dominated solutions was found during the initialisation phase by the *Pool-Init-Gen* algorithm (see Section Algorithm Section 3.1). From the results, it is apparent how the MO-NSGA-VNS provided clear advantages in terms of computing times and number of identified solutions. It is also worth clarifying that the measured CPU time needed for executing the whole *Init_phase* iterative algorithm on instances of any size was negligible as regularly less than one second. This evidence matches with the worst-case analysis already provided for the proposed *Pool-Init-Gen* algorithm, and corroborates experimentally its high performance, as it requires very limited computational resources but in some cases even produces non-dominated solutions. In order to explore and compare the quality of the solutions obtained by the two approaches, the results in Table 2 adopt the metrics (14) and (15) above described to appreciate how only a few solutions provided by MO-NSGA-VNS (around 1.25 on the average) are dominated by other solutions in the reference set, which includes the solutions found by the MO-CDP algorithm, whereas the MO-NSGA-VNS outperformed the MO-CDP approach by 25 solutions on the average. Results in Table 3 report the range of values for the three objective functions: if the Pareto front solutions span on a broader interval, then a better approximation of the reference set is reflected. These results are therefore devoted to evaluate diversity and inclusiveness of the set of obtained non-dominated solutions for each algorithm, and assess how broadly these are spread over the Pareto front. The two approaches show a quite different behaviour in this respect, as functions f_1 and f_2 span a quite wider range on most of the solved instances, with exception for the larger ones, while the range for f_3 is consistently wider for the metaheuristic approach.

Also, it is worth noticing how the lowest values in terms of cost of the CDP are regularly identified by the evolutionary metaheuristic, thus hitting one of the underlying goals of this multi-objective extension of the CDP approach: containing as much as possible the transportation costs for surveillance operations while securing the highest possible effectiveness in terms of quality of the surveillance.

The results of the experiments aimed at benchmarking MO-NSGA-VNS against standard evolutionary algorithms are provided in Table 4 by reporting, for each instance, the number NDs of solutions provided as an output by each algorithm, and the percentage $NDsRbyR$ of such solutions which result to be dominated by using the metric (15) presented above. Note such a metric is applied with a *one-vs-all* method, where each non-dominated solution set ND produced by a given algorithm is compared to a reference solution set R , being the latter equal to the union of all the solutions produced by the remaining algorithms. Clearly, such a reference set R differs from

Table 2

Results of the capacity performance measure for MO-NSGA-VNS algorithm, where the reference set R contains the solutions found by the MO-CDP algorithm. Column 1 describes the instance triplet (n, m, o) . Columns 2–3 report the NDs for algorithms MO-CDP and MO-NSGA-VNS. Columns 4–5, report $NDsbyR$ and $NDsRbyR$ metric values (Metrics (14), (15)). Last column 6 reports the solutions in common with the reference set R .

(n, m, o)	MO-CDP	MO-NSGA-VNS			
	NDs	NDs	$NDsbyR$	$NDsRbyR$	$\cap$
(40, 156, 0)	16	33	5	0.16	6
(40, 156, 1)	11	21	3	0.15	6
(40, 780, 0)	9	29	2	0.07	4
(40, 780, 1)	13	26	6	0.24	1
(40, 1404, 0)	17	29	9	0.32	2
(40, 1404, 1)	14	27	8	0.30	1
(50, 245, 0)	20	39	5	0.13	1
(50, 245, 1)	17	16	2	0.13	2
(50, 1225, 0)	9	27	1	0.04	2
(50, 1225, 1)	6	26	1	0.04	1
(50, 2205, 0)	–	22	–	–	–
(50, 2205, 1)	–	28	–	–	–
(60, 354, 0)	23	48	9	0.19	4
(60, 354, 1)	10	54	3	0.06	4
(60, 1770, 0)	–	40	–	–	–
(60, 1770, 1)	–	48	–	–	–
(60, 3186, 0)	–	31	–	–	–
(60, 3186, 1)	–	25	–	–	–
(70, 483, 0)	16	53	4	0.08	2
(70, 483, 1)	28	68	8	0.12	4
(70, 2415, 0)	–	32	–	–	–
(70, 2415, 1)	–	44	–	–	–
(70, 4347, 0)	–	22	–	–	–
(70, 4347, 1)	–	27	–	–	–
(80, 632, 0)	5	21	1	0.05	2
(80, 632, 1)	12	18	3	0.17	1
(80, 3160, 0)	–	46	–	–	–
(80, 3160, 1)	–	50	–	–	–
(80, 5688, 0)	–	14	–	–	–
(80, 5688, 1)	–	22	–	–	–
(90, 801, 0)	1	26	0	0.00	1
(90, 801, 1)	1	19	0	0.00	1
(90, 4005, 0)	–	26	–	–	–
(90, 4005, 1)	–	26	–	–	–
(90, 7209, 0)	–	21	–	–	–
(90, 7209, 1)	–	30	–	–	–

the one adopted in Table 2. It can be observed how the MO-NSGA-VNS outperforms both the competing approaches, providing somewhat regularly the highest number of solutions, and presenting at the same time the lowest values for the dominance metric. Indeed, although at a first glance on some instances the number of solutions returned by the *subpaths-NSGA* may appear higher than those provided by the MO-NSGA-VNS, $NDsRbyR$ scores are effective at showing how most of such produced solutions are dominated and thus do not represent suitable candidates to approximate the Pareto front. Figs. 5–6 are instrumental at visualising the prevailing profile of the MO-NSGA-VNS algorithm solutions as compared to the competing approaches.

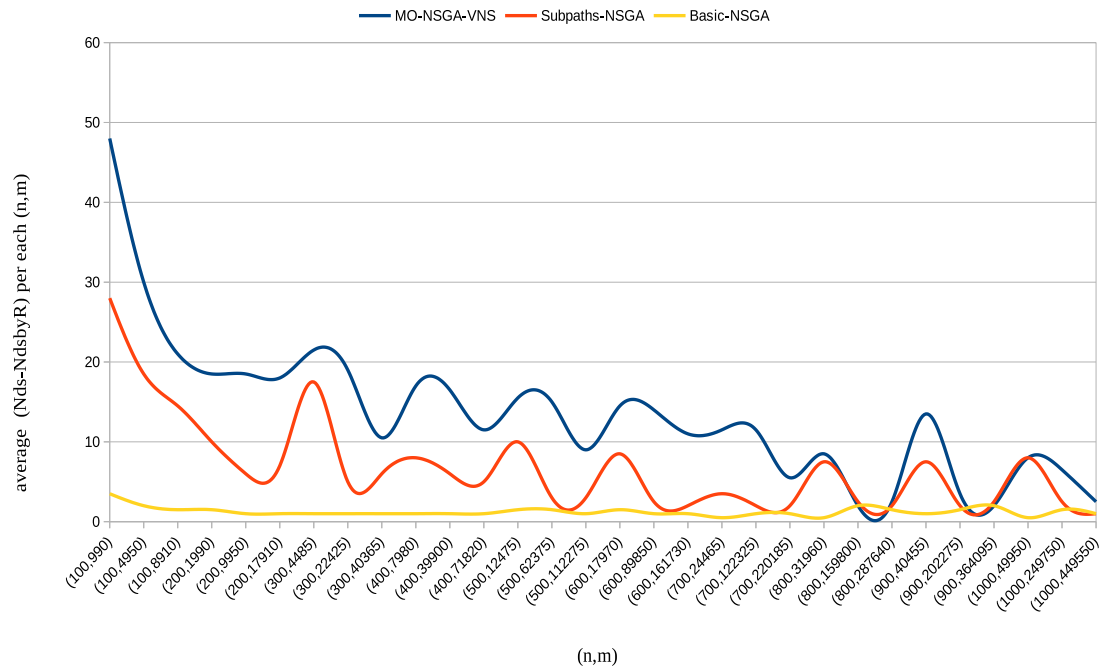
6. Conclusions

In this paper we proposed a multiple objective approach to extend the Critical Disruption Path problem, a path-based network interdiction problem aimed at optimising surveillance operations. While coupling the size of the maximal connected component and the number of such components in the effort to maximise the surveillance effectiveness, our variant of the CDP problem concurrently seeks for the minimisation of the critical path cost, thus increasing the impact of adopting a CDP approach for practical purposes, supporting surveillance planning for a variety of application fields and different transportation means. Motivated by the provably high level of complexity of the considered problem, we developed an original evolutionary metaheuristic

Table 3

Performance comparisons between MO-CDP and MO-NSGA-VNS algorithms, column 1 describes the instance triplet (n, m, o) . Columns 2–5 report the range of the three objective functions f_1, f_2, f_3 and the cost as a number of hops for the MO-CDP problem. Columns 6–9 report these ranges for the MO-NSGA-VNS problem.

(n, m, o)	MO-CDP				MO-NSGA-VNS			
	f_1	f_2	f_3	Hops	f_1	f_2	f_3	Hops
(40, 156, 0)	[1–36]	[2–19]	[24–711]	[2–22]	[4–36]	[2–17]	[24–417]	[2–15]
(40, 156, 1)	[1–30]	[3–15]	[203–827]	[7–25]	[4–30]	[3–15]	[203–587]	[7–19]
(40, 780, 0)	[1–32]	[2–7]	[122–1241]	[6–32]	[10–32]	[2–3]	[122–2313]	[6–27]
(40, 780, 1)	[1–37]	[2–7]	[134–1327]	[1–33]	[12–37]	[2–3]	[134–2825]	[1–26]
(40, 1404, 0)	[0–35]	[0–3]	[84–2440]	[4–39]	[10–35]	[1–2]	[84–9237]	[4–29]
(40, 1404, 1)	[0–36]	[0–3]	[130–2302]	[3–39]	[11–36]	[1–2]	[130–8301]	[3–27]
(50, 245, 0)	[1–36]	[11–32]	[93–970]	[3–20]	[5–36]	[11–26]	[93–944]	[3–17]
(50, 245, 1)	[1–30]	[14–32]	[73–986]	[6–21]	[10–30]	[14–22]	[73–486]	[6–17]
(50, 1225, 0)	[1–45]	[1–7]	[83–3269]	[4–45]	[21–45]	[1–2]	[83–3822]	[4–27]
(50, 1225, 1)	[1–45]	[1–7]	[166–3103]	[4–45]	[18–45]	[1–2]	[166–9588]	[4–31]
(50, 2205, 0)	[–]	[–]	[–]	[–]	[19–46]	[1–1]	[117–10107]	[3–30]
(50, 2205, 1)	[–]	[–]	[–]	[–]	[18–46]	[1–2]	[129–15299]	[3–30]
(60, 354, 0)	[1–52]	[5–30]	[88–1969]	[3–31]	[8–52]	[5–26]	[88–1897]	[3–21]
(60, 354, 1)	[1–52]	[4–27]	[135–2080]	[4–32]	[9–52]	[4–25]	[135–2638]	[4–25]
(60, 1770, 0)	[–]	[–]	[–]	[–]	[24–53]	[1–9]	[247–10659]	[6–30]
(60, 1770, 1)	[–]	[–]	[–]	[–]	[27–57]	[1–4]	[135–8213]	[2–30]
(60, 3186, 0)	[–]	[–]	[–]	[–]	[28–56]	[1–3]	[458–17497]	[3–29]
(60, 3186, 1)	[–]	[–]	[–]	[–]	[25–57]	[1–3]	[188–17405]	[2–32]
(70, 483, 0)	[1–57]	[9–34]	[239–3346]	[4–36]	[6–57]	[9–30]	[239–2533]	[4–21]
(70, 483, 1)	[1–59]	[8–33]	[219–3039]	[3–37]	[10–59]	[8–29]	[219–2436]	[3–21]
(70, 2415, 0)	[–]	[–]	[–]	[–]	[35–64]	[2–6]	[119–12206]	[4–28]
(70, 2415, 1)	[–]	[–]	[–]	[–]	[33–68]	[1–7]	[122–12359]	[1–30]
(70, 4347, 0)	[–]	[–]	[–]	[–]	[36–63]	[1–2]	[123–21697]	[6–32]
(70, 4347, 1)	[–]	[–]	[–]	[–]	[38–63]	[1–2]	[249–27361]	[5–30]
(80, 632, 0)	[3–67]	[6–35]	[402–2371]	[5–36]	[37–67]	[6–21]	[402–1537]	[5–20]
(80, 632, 1)	[1–73]	[4–47]	[64–3133]	[3–36]	[39–73]	[4–23]	[64–359]	[3–14]
(80, 3160, 0)	[–]	[–]	[–]	[–]	[48–76]	[1–11]	[312–18037]	[3–22]
(80, 3160, 1)	[–]	[–]	[–]	[–]	[40–75]	[1–11]	[165–15957]	[4–32]
(80, 5688, 0)	[–]	[–]	[–]	[–]	[51–71]	[1–1]	[280–12378]	[8–28]
(80, 5688, 1)	[–]	[–]	[–]	[–]	[48–70]	[1–2]	[210–18573]	[9–30]
(90, 801, 0)	[86–86]	[1–1]	[170–170]	[3–3]	[54–86]	[1–1]	[170–4218]	[3–35]
(90, 801, 1)	[88–88]	[1–1]	[60–60]	[1–1]	[61–88]	[1–1]	[60–2008]	[1–28]
(90, 4005, 0)	[–]	[–]	[–]	[–]	[63–84]	[1–2]	[451–21072]	[5–25]
(90, 4005, 1)	[–]	[–]	[–]	[–]	[57–83]	[3–7]	[159–18480]	[4–27]
(90, 7209, 0)	[–]	[–]	[–]	[–]	[61–83]	[1–2]	[261–44696]	[5–28]
(90, 7209, 1)	[–]	[–]	[–]	[–]	[62–87]	[1–2]	[288–45496]	[2–26]

**Fig. 5.** Comparisons among the different evolutionary algorithms reporting $N Ds - N DsbyR$ values for each pair (n, m) .

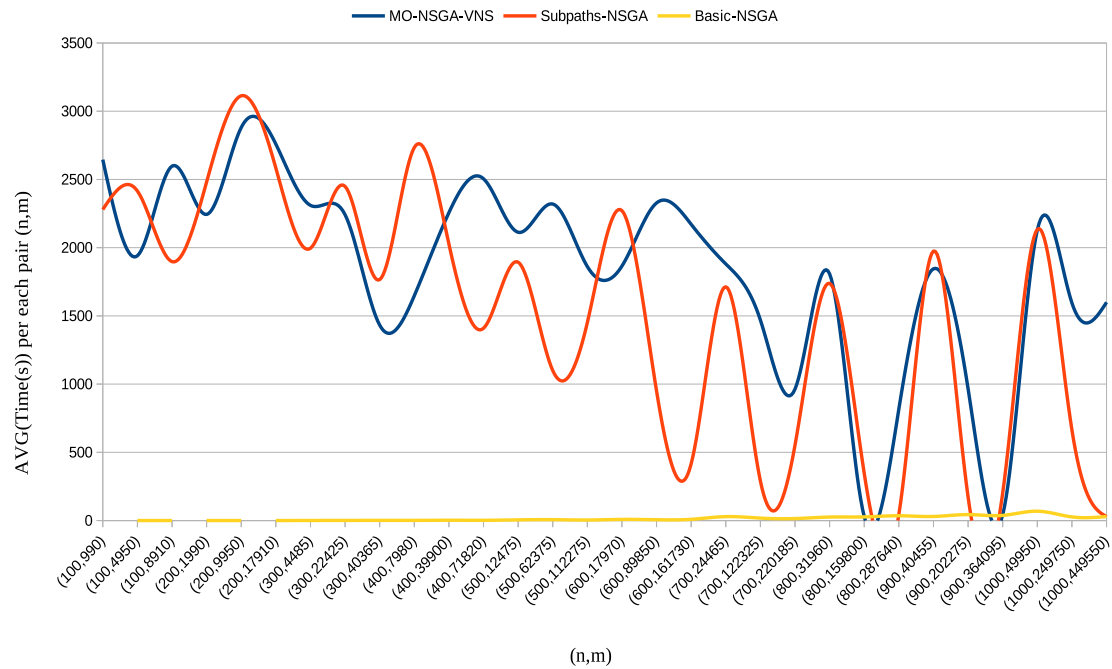


Fig. 6. Comparisons among the different evolutionary algorithms reporting average time values for each pair (n, m) .

Table 4

Results of the capacity performance measure for all the created algorithms, where the reference set R is the union of all the ND sets found by the other algorithms. Column 1 describes the instance triplet (n, m, o) . Columns 2–3, 4–5 and 6–7 report the NDs value and the $NDsRbyR$ metric value (see Metric (15)) for each genetic algorithm.

(n, m, o)	MO-NSGA-VNS		Subpaths-NSGA		Basic-NSGA	
	NDs	NDsRbyR	NDs	NDsRbyR	NDs	NDsRbyR
(40, 156, 0)	33	0.15	35	0.29	3	0.33
(40, 156, 1)	21	0.14	20	0.35	4	0.50
(40, 780, 0)	29	0.07	30	0.70	1	1.00
(40, 780, 1)	26	0.23	23	0.83	4	0.25
(40, 1404, 0)	29	0.31	26	0.77	2	0.50
(40, 1404, 1)	27	0.30	35	0.66	2	1.00
(50, 245, 0)	39	0.21	37	0.35	7	0.71
(50, 245, 1)	16	0.19	38	0.29	5	0.40
(50, 1225, 0)	27	0.04	44	0.64	3	0.33
(50, 1225, 1)	26	0.04	32	0.66	3	0.67
(50, 2205, 0)	22	0.00	30	0.60	2	0.50
(50, 2205, 1)	28	0.21	27	0.48	2	1.00
(60, 354, 0)	48	0.31	50	0.32	5	0.80
(60, 354, 1)	54	0.17	44	0.41	4	1.00
(60, 1770, 0)	40	0.02	58	0.36	4	0.50
(60, 1770, 1)	48	0.04	50	0.48	6	0.50
(60, 3186, 0)	31	0.00	39	0.56	4	0.50
(60, 3186, 1)	25	0.00	36	0.47	4	0.75
(70, 483, 0)	53	0.15	53	0.45	6	0.50
(70, 483, 1)	68	0.25	48	0.38	6	0.67
(70, 2415, 0)	32	0.03	50	0.40	6	0.33
(70, 2415, 1)	44	0.00	43	0.56	5	0.20
(70, 4347, 0)	22	0.00	23	0.61	3	0.67
(70, 4347, 1)	27	0.00	23	0.74	2	1.00
(80, 632, 0)	21	0.19	37	0.19	7	0.86
(80, 632, 1)	18	0.17	51	0.24	4	1.00
(80, 3160, 0)	46	0.04	36	0.22	6	0.67
(80, 3160, 1)	50	0.06	47	0.19	6	0.50
(80, 5688, 0)	14	0.00	35	0.31	3	0.67
(80, 5688, 1)	22	0.00	31	0.32	3	0.67
(90, 801, 0)	26	0.00	25	0.80	3	1.00
(90, 801, 1)	19	0.00	26	0.58	4	0.25
(90, 4005, 0)	26	0.00	34	0.59	4	0.75
(90, 4005, 1)	26	0.00	24	0.25	6	0.50
(90, 7209, 0)	21	0.00	23	0.43	2	0.50
(90, 7209, 1)	30	0.03	27	0.56	3	0.67

algorithmic approach, which hybridises modified-NSGA-II and VNS for finding efficiently an approximation of the Pareto front on increasing size networks. We also complemented this algorithm with a tailored preliminary procedure, based on a minimum variant of the shortest disjoint path pair problem, and aimed at allowing a preliminary efficient calculation of high quality initial solutions to feed the evolutionary algorithm, concurrently decreasing its computational effort requirements by reducing the set of candidates.

Data availability

Data will be made available on request.

References

- Albert, R., Jeong, H., Barabasi, A., 2000. Error and attack tolerance of complex networks. *Nature* 406, 378–382.
- Arroyo, J., dos Santos Ottoni, R., de Paiva Oliveira, A., 2011. Multi-objective variable neighborhood search algorithms for a single machine scheduling problem with distinct due windows. *Electron. Notes Theor. Comput. Sci.* 281, 5–19, Proceedings of the 2011 Latin American Conference in Informatics (CLEI).
- Arulselvan, A., Commander, C., Eleftheriadou, L., Pardalos, P.M., 2009. Detecting critical nodes in sparse graphs. *Comput. Oper. Res.* 36 (7), 2193–2200.
- Arulselvan, A., Commander, C.W., Pardalos, P., Shylo, O., 2007. Managing network risk via critical node identification. In: Rustem, B., Gulpinar, N. (Eds.), *Risk Management in Telecommunication Networks*. Springer.
- Asefi, H., Jolai, F., Rabiee, M., Araghi, M.T., 2014. A hybrid NSGA-II and VNS for solving a bi-objective no-wait flexible flowshop scheduling problem. *Int. J. Adv. Manuf. Technol.* 75 (5–8), 1017–1033.
- Ball, M., Golden, B., Vohra, R., 1989. Finding the most vital arcs in a network. *Oper. Res. Lett.* 8 (2), 73–76.
- Basseur, M., Talbi, E.-G., Nebro, A., Alba, E., 2006. Avancées des métaheuristiques pour l'optimisation combinatoire multi-objectif. Technical report, INRIA research report.
- Cohen, R., Havlin, S., Ben-Avraham, D., 2003. Efficient immunization strategies for computer networks and populations. *Phys. Rev. Lett.* 91 (24), 247–901.
- Deb, K., Jain, H., 2014. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints. *IEEE Trans. Evol. Comput.* 18 (4), 577–601.
- Deb, K., Pratap, A., Agarwal, S., Meyarivan, T., 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* 6 (2), 182–197.
- Dinh, T., Xuan, Y., Thai, M., Park, E., Znati, T., 2010. On approximation of new optimization methods for assessing network vulnerability. In: *Proceedings of the 29th Conference on Information Communications. INFOCOM '10*, IEEE Press, Piscataway, NJ, USA, pp. 2678–2686.

- Eilam-Tzoref, T., 1998. The disjoint shortest paths problem. *Discrete Appl. Math.* 85 (2), 113–138.
- Emmerich, M., Deutz, A., 2018. A tutorial on multiobjective optimization: fundamentals and evolutionary methods. *Nat. Comput.* 17 (3), 585–609.
- Emmerich, M., Hochstrate, N., Naujoks, B., 2005. An EMO algorithm using the Hypervolume Measure as Selection Criterion. In: *International Conference on Evolutionary Multi-Criterion Optimization*. Springer, pp. 62–76.
- Fortune, S., Hopcroft, J., Wyllie, J., 1980. The directed subgraph homeomorphism problem. *Theoret. Comput. Sci.* 10 (2), 111–121.
- Frank, A., 1988. Packing Paths, Circuits and Cuts: A Survey. In: *Report, Sonderforschungsbereich 303*.
- Furini, F., Ljubić, I., Martin, S., San Segundo, P., 2019. The maximum clique interdiction problem. *European J. Oper. Res.* 277 (1), 112–127.
- Gandibleux, X., Ehrgott, M., 2005. 1984–2004–20 Years of multiobjective metaheuristics. but what about the solution of combinatorial problems with multiple objectives? In: *Evolutionary Multi-Criterion Optimization*. Springer, pp. 33–46.
- Geiger, M.J., 2008. Randomised variable neighbourhood search for multi objective optimisation. *CoRR arXiv:0809.0271*.
- Granata, D., Sgalambro, A., 2016. Network interdiction through length-bounded critical disruption paths: a bi-objective approach. *Electron. Notes Discrete Math.*
- Granata, D., Steeger, G., Rebennack, S., 2013. Network interdiction via a critical disruption path: Branch-and-price algorithms. *Comput. Oper. Res.* 40 (11), 2689–2702.
- Hansen, P., Mladenović, N., 1997. Variable neighborhood search for the p-median. *Locat. Sci.* 5 (4), 207–226.
- Harris, T., Ross, F., 1955. In: *Research Memorandum RM-1573 (Ed.), Fundamentals of a Method for Evaluating Rail Net Capacities*. Research Memorandum RM-1573, The Rand Corporation, Santa Monica, CA.
- Holme, P., Kim, B., Yoon, C., Han, S., 2002. Attack vulnerability of complex networks. *Phys. Rev. E* 65, 056109.
- Ishibuchi, H., Murata, T., 1996. Multi-objective genetic local search algorithm. In: *Evolutionary Computation, 1996., Proceedings of IEEE International Conference on*. pp. 119–124.
- Ishibuchi, H., Murata, T., 1998. A multi-objective genetic local search algorithm and its application to flowshop scheduling. *Syst., Man, Cybern., Part C: Appl. Rev., IEEE Trans.* 28 (3), 392–403.
- Israeli, E., Wood, R.K., 2002. Shortest-path network interdiction. *Networks* 40, 97–111.
- Jozefowiez, N., Semet, F., Talbi, E., 2008. From single-objective to multi-objective vehicle routing problems: Motivations, case studies, and methods. In: *Golden, B., Raghavan, S., Wasil, E. (Eds.), The Vehicle Routing Problem: Latest Advances and New Challenges*. In: *Operations Research & Computer Science Interfaces*, vol. 43, Springer US, pp. 445–471.
- Lalou, M., M.A., T., H., K., 2018. The critical node detection problem in networks: A survey. *Comp. Sci. Rev.* 28, 92–117.
- Li, C., McCormick, S., Simchi-Levi, D., 1975. On the computational complexity of combinatorial problems. *Networks* 5, 45–68.
- Li, C., McCormick, S., Simchi-Levi, D., 1990. The complexity of finding two disjoint paths with min-max objective function. *Discrete Appl. Math.* 26 (1), 105–115.
- Lim, C., Smith, J., 2007. Algorithms for discrete and continuous multicommodity flow network interdiction problems. *IEE Trans.* 39 (1), 15–26.
- Mahdavi, P., Boginski, V., Pasiliao, E., 2014. Minimum vertex blocker clique problem. *Networks* 64 (1), 48–64.
- Martins, L., Gomes, T., Tipper, D., 2017. Efficient heuristics for determining node-disjoint path pairs visiting specified nodes. *Networks* 70 (4), 292–307.
- Mishkovski, I., Biey, M., Kocarev, L., 2011. Vulnerability of complex networks. *Commun. Nonlinear Sci. Numer. Simul.* 16 (1), 341–349.
- Mladenović, N., Hansen, P., 1997. Variable neighborhood search. *Comput. Oper. Res.* 24 (11), 1097–1100.
- Polacek, M., Hartl, R., Doerner, K., Reimann, M., 2004. A variable neighborhood search for the multi depot vehicle routing problem with time windows. *J. Heuristics* 10 (6), 613–627.
- Royset, J., Wood, R., 2007. Solving the bi-objective maximum-flow network-interdiction problem. *INFORMS J. Comput.* 19 (2), 175–184.
- Schilde, M., Doerner, K., Hartl, R., Kiechle, G., 2009. Metaheuristics for the bi-objective orienteering problem. *Swarm Intell.* 3 (3), 179–201.
- Shen, S., Smith, J., Goli, R., 2012. Exact interdiction models and algorithms for disconnecting networks via node deletions. *Discrete Optim.* 9 (3), 172–188.
- Shiloach, Y., 1980. A polynomial solution to the undirected two paths problem. *J. ACM* 27 (3), 445–456.
- Smith, J., Song, Y., 2020. A survey of network interdiction models and algorithms. *European J. Oper. Res.* 283 (3), 797–811.
- Summa, M., Grosso, A., Locatelli, M., 2011. Complexity of the critical node problem over trees. *Comput. Oper. Res.* 38 (12), 1766–1774.
- Suurballe, J., 1974. Disjoint paths in a network. *Networks* 4 (2), 125–145.
- Suurballe, J., Tarjan, R., 1984. A quick method for finding shortest pairs of disjoint paths. *Networks* 14 (2), 325–336.
- Talbi, E.-G., Basseur, M., Nebro, A., Alba, E., 2012. Multi-objective optimization using metaheuristics: non-standard algorithms. *Int. Trans. Oper. Res.* 19 (1–2), 283–305.
- Tragoudas, S., Varol, Y., 1997. Computing disjoint paths with length constraints. In: *d'Amore, F., Franciosa, P., Marchetti-Spaccamela, A. (Eds.), Graph-Theoretic Concepts in Computer Science*. In: *Lecture Notes in Computer Science*, vol. 1197, Springer Berlin Heidelberg, pp. 375–389.
- Ventresca, M., 2012. Global search algorithms using a combinatorial unranking-based problem representation for the critical node detection problem. *Comput. Oper. Res.* 39 (11), 2763–2775.
- Ventresca, M., Aleman, D., 2014a. A derandomized approximation algorithm for the critical node detection problem. *Comput. Oper. Res.* 43, 261–270.
- Ventresca, M., Aleman, D., 2014b. A fast greedy algorithm for the critical node detection problem. In: *Zhang, Z., Wu, L., Xu, W., Du, D.-Z. (Eds.), Combinatorial Optimization and Applications*. In: *Lecture Notes in Computer Science*, vol. 8881, Springer International Publishing, pp. 603–612.
- Vygen, J., 1994. Disjoint Paths. *Citeseer*.
- Walteros, J., Veremyev, A., Pardalos, P., Pasiliao, E., 2019. Detecting critical node structures on graphs: A mathematical programming approach. *Networks* 73 (1), 48–88.
- Wollmer, R., 1964. Removing arcs from a network. *Oper. Res.* 12 (6), 934–940.
- Wood, R., 1993. Deterministic network interdiction. *Math. Comput. Modelling* 17 (2), 1–18.
- Zavala, G., Nebro, A., Luna, F., Coello Coello, C., 2014. A survey of multi-objective metaheuristics applied to structural optimization. *Struct. Multidiscip. Optim.* 49 (4), 537–558.
- Zenklus, R., 2010. Matching interdiction. *Discrete Appl. Math.* 158 (15), 1676–1690.
- Zhang, Q., Li, H., 2007. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Trans. Evol. Comput.* 11 (6), 712–731.
- Zhou, T., Fu, Z.F., Wang, B., 2006. Epidemic dynamics on complex networks. *Prog. Natural Sci.* 16, 452.
- Zitzler, E., Laumanns, M., Bleuler, S., 2004. A tutorial on evolutionary multiobjective optimization. In: *Metaheuristics for Multiobjective Optimisation*. Springer, pp. 3–37.
- Zitzler, E., Laumanns, M., Thiele, L., 2001. SPEA2: Improving the Strength Pareto Evolutionary Algorithm. Technical report.